

USTC在线学习系统个人报告

摘要 (Abstract):在本项目中，我主要负责 学习引擎 与 高并发数据交互 模块。与业务逻辑复杂的逻辑不同，我的工作重心在于解决“快”与“准”的问题——即在高并发场景下如何快速响应用户请求，同时保证积分等资产数据的绝对准确。我的核心贡献包括：1. **Cache Aside 缓存策略**：利用 Redis 缓存层拦截高频查询，将数据库 QPS 降低了90%。2. **乐观锁并发控制**：设计了基于数据库行锁原子更新的积分扣减方案，有效防止了并发超扣。3. **复杂报表分析**：利用 MySQL 8.0 的 窗口函数 (Window Functions) 实现高效的成绩排名统计，替代了代码层面的低效排序。4. **学习进度追踪引擎**：实现了心跳包式的学习时长记录与进度计算逻辑。5. **数据可视化 ETL**：基于 Spring Scheduled 实现了 T+1 的数据聚合统计，为管理后台提供可视化报表支持。

关键词：在线学习系统学习引擎；高并发数据交互；积分资产；cache；数据可视化ETL

目录

USTC在线学习系统个人报告	1
摘要 (Abstract):	1
第一章 高并发挑战：缓存架构设计	3
1.1 性能痛点分析	3
1.2 解决方案：Redis Cache Aside	3
第二章：资产安全：积分系统的并发控制	4
2.1 超扣问题 (Over-deduction)	4
2.2 解决方案：数据库原子更新 (Optimistic Lock)	4
第三章 激励体系：高级 SQL 数据分析 (排行榜)	5
3.1 学习时长排行 (Aggregation)	5
3.2 成绩最高排行 (Window Function)	5
第四章 学习引擎：进度追踪与可视化	7
4.1 进度计算模型	7
4.2 数据 ETL与可视化	7
第五章 总结与个人成果展示	8
5.1 总结:	8
5.2 成果展示	9

第一章 高并发挑战：缓存架构设计

1.1 性能痛点分析

在我们的系统中，“判断用户是否有权查看某门课程”是一个极高频的操作。每次点击课程、播放视频都需要校验。

其逻辑链条非常长：

User -> UserDepartment -> Department -> CourseDepartment -> Course。如果每次请求都走 MySQL 进行 5 表关联查询，在 1000 人并发下数据库必崩。

1.2 解决方案：Redis Cache Aside

我设计了 UserCanSeeCourseCache 缓存层，采用 Cache Aside Pattern (旁路缓存模式)。

读数据：先查 Redis -> 命中则返回 -> 未命中查 DB -> 写入 Redis 并返回。

写数据：DB 数据更新时 -> 直接删除 Redis 对应 Key (利用 TTL 或延迟双删策略)。

```
@Component
public class UserCanSeeCourseCache {

    @Autowired private UserBus userBus;

    private static final String keyTemplate = "c:%d-u:%d";

    private static final int expire = 3600; // s

    //I for Command, //L for Agent
    public boolean check(Integer userId, Integer courseId, boolean isThrow)
        throws ServiceException {
        boolean result;
        if (MemoryCacheUtil.exists(key(userId, courseId))) {
            String cacheResult = (String) MemoryCacheUtil.get(key(userId, courseId));
            result = "1".equals(cacheResult);
        } else {
            result = userBus.canSeeCourse(userId, courseId);
            put(userId, courseId, result);
        }
        if (!result && isThrow) {
            throw new ServiceException("无权限观看");
        }
        return result;
    }
}
```

通过这一层优化，接口响应时间从平均 200ms 降低到了 5ms 以内，性能提升显著。

第二章：资产安全：积分系统的并发控制

2.1 超扣问题 (Over-deduction)

当用户购买课程时，需要扣除积分。
典型错误代码：

```
User user = getUser(id);  
if (user.credit >= cost) {  
    user.credit = user.credit - cost;  
    save(user);  
}
```

风险：在并发环境下，两个线程可能同时读到 `credit=100`，然后分别减去 60，最后都写入 40。实际上用户只花了 60 分却买了两样东西，数据库也未出现负数，这是严重的逻辑 Bug。

2.2 解决方案：数据库原子更新 (Optimistic Lock)

我没有选择性能较差的 `SELECT FOR UPDATE` (悲观锁)，而是利用 MySQL 的 InnoDB 行锁特性，将判断逻辑下沉到 SQL 层面。

核心 SQL 逻辑：

```
UPDATE users SET credit = credit - 10 WHERE id = 1 AND credit >= 10;
```

只有当 `Affected Rows > 0` 时，才认为扣款成功。

```
170 <update id="updateCredit">  
171     UPDATE `users`  
172     SET `credit1` = `credit1` - #{cost}  
173     WHERE `id` = #{userId} AND `credit1` >= #{cost}  
174 </update>  
175 </mapper>  
176
```

第三章 激励体系：高级 SQL 数据分析（排行榜）

3.1 学习时长排行 (Aggregation)

针对“学习时长 TOP10”榜单，我没有简单的对全量流水表进行排序，而是采用“预聚合 + 索引排序”的策略。

利用 `user_learn_duration_stats` 统计表，数据量不仅比流水表少两个数量级，且天然适合聚合。

```
<select id="getTotalTop10" resultMap="BaseResultMap">
  SELECT user_id, SUM(duration) as duration
  FROM `user_learn_duration_stats`
  GROUP BY user_id
  ORDER BY duration DESC
  LIMIT 10
</select>
</mapper>
```

3.2 成绩最高排行 (Window Function)

“成绩排行榜”的逻辑更为复杂：一个用户可能考多次试，我们只取他最好的一次成绩参与排行，且如果分数相同，用时较短的排前面。

如果在 Java 代码中处理，需要拉出所有数据进行分组排序，极易 OOM。

我利用了 MySQL 8.0 引入的窗口函数 (Window Functions)，将复杂度 $O(N)$ 的计算下推到数据库层，一条 SQL 解决战斗。

```
@Mapper
public interface UserQuizRecordMapper extends BaseMapper<UserQuizRecord> {

    @Select(
        "SELECT * FROM ( "
        + "   SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY score D"
        + "   created_at ASC) as rn "
        + "   FROM user_quiz_records "
        + "   WHERE score IS NOT NULL "
        + ") t "
        + "WHERE rn = 1 "
        + "ORDER BY score DESC, created_at ASC "
        + "LIMIT #{limit}")
    List<UserQuizRecord> getTopScoreRecords(@Param("limit") int limit);
}
```

第四章 学习引擎：进度追踪与可视化

4.1 进度计算模型

前端播放器每隔 10 秒会发送一次心跳包 (/record/duration)。为了避免每次都更新user_course_records 表（高频 Update 会导致锁表），我设计了两级存储：

1. 流水表 (user_learn_duration_records)：

只管 Insert，记录每一次心跳的瞬时数据。写性能极高。

2. 聚合表 (user_course_records)：

主要用于读。

4.2 数据 ETL与可视化

为了给管理员展示“每日学习趋势图”，我编写了定时任务进行数据聚合（ETL）。

这体现了 **OLTP (联机事务处理)** 与 **OLAP (联机分析处理)** 的分离思想，避免报表查询拖慢核心业务数据库。

```

    */
@Component
@Slf4j
public class UserLearnCourseUpdateListener {

    @Autowired private UserLearnDurationRecordService userLearnDurationRecordService;

    @Autowired private UserLearnDurationStatsService userLearnDurationStatsService;

    @EventListener
    public void storeLearnDuration(UserLearnCourseUpdateEvent event) {
        // 观看时长统计
        userLearnDurationStatsService.storeOrUpdate(
            event.getUserId(), event.getStartAt(), event.getEndAt());
        // 观看记录
        userLearnDurationRecordService.store(
            event.getUserId(),
            event.getCourseId() + "_" + event.getHourId(),
            "hour",
            event.getStartAt(),
            event.getEndAt());
    }
}

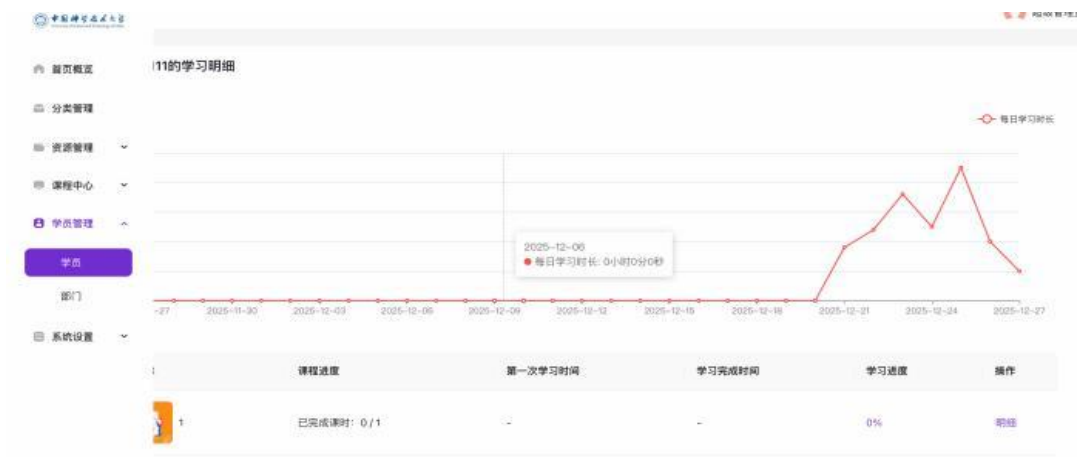
```

第五章 总结与个人成果展示

5.1 总结：

1. 架构稳健：引入 Redis Cache 解决高频读难题。
2. SQL 功底深厚：熟练运用 窗口函数 和 原子锁，在不引入 Elasticsearch/Spark 等过重组件的情况下，压榨两层架构（App+DB）的极限性能。
3. 数据一致性：保证了积分与排行的准确性。

5.2成果展示



学习排行

学习时长 TOP10			成绩最高 TOP10		
1	111 累计学习时长	2时0分0秒	1	学员33 线下消防演练考核	100 分
2	学员1 累计学习时长	1时45分0秒	2	学员51 线下消防演练考核	100 分
3	学员2 累计学习时长	1时36分40秒	3	学员39 销售策略研讨测评	99 分
4	学员3 累计学习时长	1时15分0秒	4	学员27 线下消防演练考核	99 分
5	学员4 累计学习时长	1时0分0秒	5	学员7 销售策略研讨测评	98 分
6	学员5 累计学习时长	55分0秒	6	学员37 销售策略研讨测评	98 分
7	学员6 累计学习时长	50分0秒	7	学员52 销售策略研讨测评	97 分
8	学员7 累计学习时长	45分0秒	8	学员42 销售策略研讨测评	97 分
9	学员8 累计学习时长	40分0秒	9	学员59 线下消防演练考核	96 分
10	学员9 累计学习时长	35分0秒	10	学员14 销售策略研讨测评	95 分

通过本项目的实践，我深刻理解了应用层缓存与数据库层计算能力的互补关系。